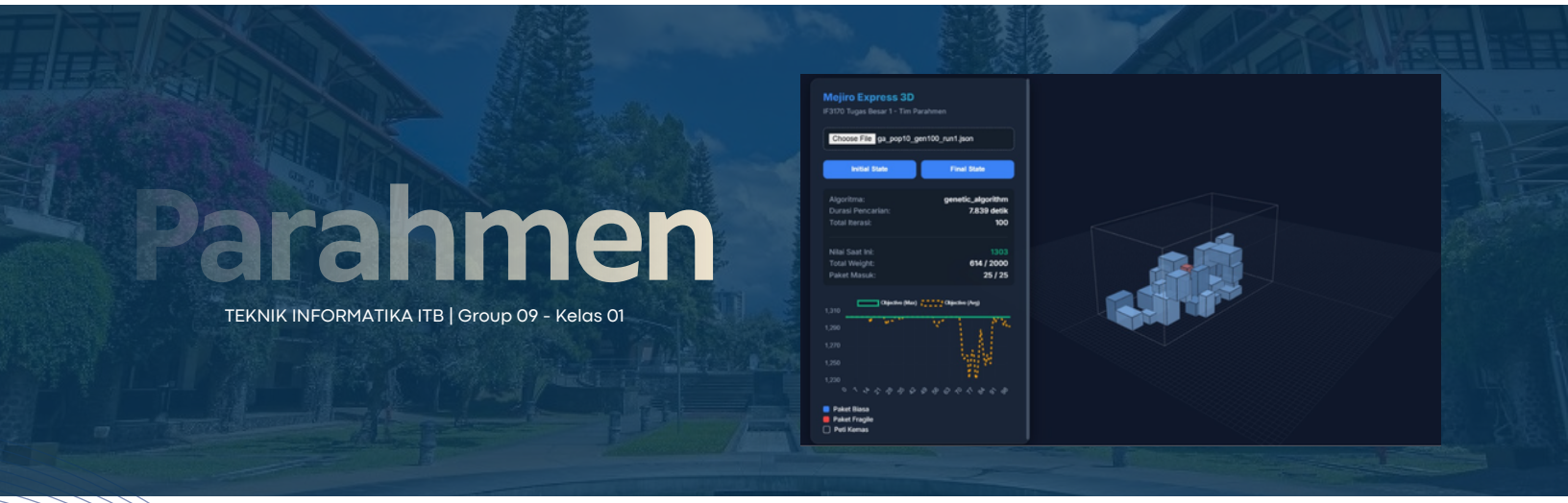
LAPORAN TUGAS BESAR 1

IF3170 Inteligensi Artifisial 2026/2027



Disusun oleh:

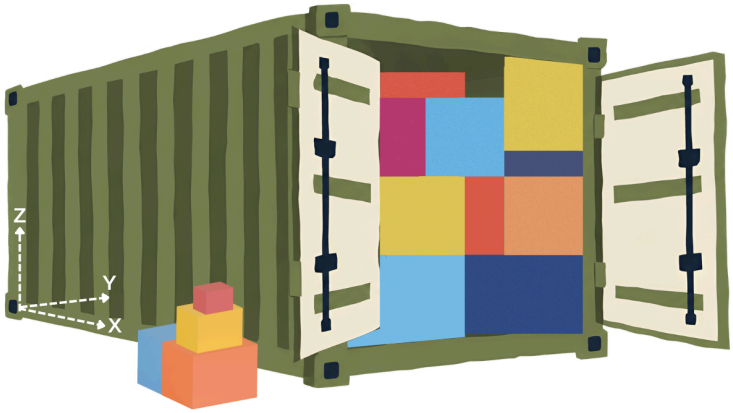
Ardell Aghna Mahendra (NIM 13523**151**)
Fazri Arrashyi Putra (13524**127**)
Dzakwan Muhammad K. P. P. (13524**145**)

DAFTAR ISI

DAFTAR ISI I
1. Deskripsi Persoalan **1**
2. Pembahasan dan Analisa **3**
2.1. Kode Algoritma 3
2.1.1. Representasi Model (src/model) 3
2.1.2. Constraint dan Fungsi Objektif (src/constraint dan src/objective) 5
2.1.3. Operator Pergerakan dan Inisialisasi (src/moves dan src/init) 7
2.1.4. Hill Climbing (src/algortima/hill_climbing) 8
2.1.5. Simulated Annealing (src/algoritma/simulated_annealing) 10
2.1.6. Genetic Algorithm (src/algoritma/genetic_algorithm) 11
2.1.7. Sistem Runner dan Konfigurasi (src/main dan src/config) 12
2.2. Hasil Eksperimen 13
2.2.1. Sistem Runner dan Konfigurasi (src/main dan src/config) 13
2.2.2. Hasil Eksperimen Genetic Algorithm 15
2.3. Analisa Hasil Eksperimen 17
2.3.1. Ringkasan Performa per Algoritma 17
2.3.2. Analisis Kualitas Solusi 18
2.3.3. Analisis Durasi dan Tradeoff Kualitas Waktu 19
2.3.4. Analisis Probabilitas pada Simulated Annealing 20
2.3.5. Analisis Pengaruh Parameter pada Genetic Algorithm 21
2.3.6. Analisis Pengaruh Parameter pada Genetic Algorithm 22
3. Kesimpulan dan Saran **23**
3.1. Kesimpulan 23
3.2. Saran 26
4. Lampiran **27**
4.1. Surat Kebijakan Penggunaan AI 27
4.2. Detail Pembagian Tugas 28
4.3. Lampiran Lainnya 28
5. Referensi **29**

1. Deskripsi Persoalan

Dalam dunia logistik dan distribusi barang, efisiensi pemanfaatan ruang angkut merupakan faktor penting yang berpengaruh langsung pada biaya operasional. Truk peti kemas memiliki ruang dan kapasitas berat yang terbatas, sementara paket yang harus diangkut sangat beragam, baik dari segi ukuran, berat, nilai, maupun tingkat kerapuhannya. Persoalan menyusun paket-paket tersebut ke dalam ruang tiga dimensi (3D) agar menghasilkan keuntungan terbesar dikenal dengan istilah *3D Truck Loading Problem*[1]. *Simplenya* persoalan ini dapat diibaratkan seperti permainan menyusun balok pada ruang terbatas, tetapi dengan aturan fisik yang harus dipatuhi dan tujuan untuk memaksimalkan total nilai paket yang terangkut.



Gambar 1.1. Representasi persoalan 3D Truck Loading: paket-paket berwarna ditempatkan ke dalam ruang peti kemas tiga dimensi, sementara sebagian paket berada di luar truk karena keterbatasan ruang dan kapasitas.

Permasalahan utama dalam persoalan ini terletak pada banyaknya kemungkinan susunan yang harus dipertimbangkan. Setiap paket dapat diletakkan pada posisi yang berbeda, diputar ke beberapa orientasi, atau bahkan tidak dimuat sama sekali. Apabila kita mencoba seluruh kemungkinan susunan secara menyeluruh (*brute force*) akan memakan waktu yang sangat besar seiring bertambahnya jumlah paket. Di sisi lain, setiap susunan harus memenuhi sejumlah aturan, yaitu paket tidak boleh melayang atau menembus dinding truk, barang pecah-belah tidak boleh tertindih, total berat tidak boleh melebihi kapasitas truk, dan tidak ada paket yang saling tumpang tindih. Untuk itu, diperlukan algoritma yang bisa menyelesaikan permasalahan ini tanpa harus mengecek seluruh kemungkinan.

Dalam rangka memenuhi Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial, penulis mengembangkan sebuah program untuk mencari konfigurasi pemuatan paket yang optimal secara otomatis. Program ini mengimplementasikan

algoritma *local search*, yaitu paradigma pencarian yang bekerja dengan memperbaiki satu solusi secara bertahap melalui perubahan-perubahan kecil pada solusi tersebut[2]. Pendekatan ini sudah banyak digunakan pada berbagai persoalan optimasi, seperti contoh pada penjadwalan, perencanaan rute, hingga penyusunan tata letak[3]. Pada program ini, tiga algoritma diimplementasikan dan dibandingkan, yaitu *Hill Climbing* beserta variasinya, *Simulated Annealing* yang memungkinkan pencarian keluar dari *local optima*, dan *Genetic Algorithm* yang memanfaatkan sekumpulan solusi sekaligus. Hasil pencarian dari ketiga algoritma tersebut juga dapat divisualisasikan secara tiga dimensi.

Tujuan yang ingin dicapai melalui tugas dan penyusunan laporan ini adalah:

1. Merumuskan persoalan *3D Truck Loading* ke dalam bentuk persoalan optimasi yang dapat diselesaikan dengan algoritma *local search*.
2. Mengimplementasikan algoritma *Hill Climbing*, *Simulated Annealing*, dan *Genetic Algorithm* untuk mencari konfigurasi pemuatan dengan total nilai terbesar.
3. Menganalisis dan membandingkan kinerja ketiga algoritma berdasarkan nilai akhir yang dicapai, perkembangan nilai selama pencarian, dan durasi prosesnya.

Dengan begitu, harapannya laporan dan program ini dapat menjelaskan atau menggambarkan bagaimana algoritma *local search* mampu menyelesaikan persoalan pemuatan paket yang kompleks secara lebih efisien dan terukur.

Footnote Information:

[1] **Tim Asisten IF3170,** "*Spesifikasi Tugas Besar 1 IF 3170 Inteligensi Artifisia*l", Program Studi Teknik Informatika Institut Teknologi Bandung", 2026.
[2] **S. Russell dan P. Norvig,** "*Artificial Intelligence: A Modern Approach*, edisi ke-4, Pearson", 2020.
[3] **Tim Pengajar IF3170,** *"Local Search: Bahan Kuliah IF3170 Intelegensi Artifisial"*, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

2. Pembahasan dan Analisa

Pada bagian ini akan dibahas implementasi ketiga algoritma local search yang digunakan untuk menyelesaikan persoalan 3D Truck Loading Optimization, beserta hasil dan analisis eksperimen yang telah dijalankan.

2.1. Kode Algoritma

2.1.1. Representasi Model (src/model)

Fondasi dari seluruh sistem dibangun di atas empat kelas model utama. Kelas Package (package.py) merepresentasikan paket sebagai balok tiga dimensi. Setiap paket menyimpan dimensi asli (og_dim), dimensi efektif setelah rotasi (cur_dim), label orientasi (orient) yang memetakan sisi paket ke sumbu kartesian, serta atribut val (value), w (weight), is_fragile, dan eta. Terdapat 6 orientasi valid yang dihasilkan dari semua permutasi sumbu {x, y, z}, didefinisikan sebagai konstanta VALID_ORIENTATIONS. Konstruktor mendukung parameter posisional maupun keyword untuk fleksibilitas, serta melakukan validasi orientasi di awal konstruksi. Paket mendukung operasi rotate(axis) yang menukar dua sisi sesuai sumbu rotasi berdasarkan dictionary ROTATION_SWAP, dan set_orientation(orientation) yang mengatur orientasi secara eksplisit sekaligus menghitung ulang dimensi efektif. Serialisasi ke JSON dilakukan melalui to_dict() dan deserialisasi melalui from_dict().

```
class Package:
    def __init__(self, pkg_id: str, dim=None, val=0, w=0, is_fragile=False,
eta=0, *,
                 dimensions=None, value=None, weight=None, position=None,
orientation=None):
        self.id: str = pkg_id
        self.og_dim: Dict[str, int] = dict(dim)
        self.cur_dim: Dict[str, int] = dict(dim)
        self.val: float = value if value is not None else val
        self.w: float = weight if weight is not None else w
        self.is_fragile: bool = is_fragile
        self.eta: int = eta
        self.orient: Dict[str, str] = (
            dict(orientation) if orientation is not None else {"w": "x",
"l": "y", "h": "z"}
        )
        self.position: Optional[Tuple[int, int, int]] = position
```

Kelas Truck (truck.py) merepresentasikan peti kemas dengan dimensi {w, l, h} dan kapasitas angkut maksimum max_cap. Kelas ini menyediakan properti w, l, h untuk akses langsung ke dimensi, serta method copy(), to_dict(), dan from_dict().

```
class Truck:
    def __init__(self, dim=None, max_cap=0, *, dimensions=None,
max_capacity=None):
        self.dim: Dict[str, int] = dict(dim)
        self.max_cap: float = max_capacity if max_capacity is not None else
max_cap
```

Kelas State (state.py) menyatukan truk dan seluruh paket ke dalam satu konfigurasi penempatan. Dictionary placements memetakan setiap id paket ke koordinat tiga dimensi (x, y, z) atau None jika paket berada di luar truk. Pada konstruksi, posisi setiap paket disinkronkan dengan dictionary placements. State menyediakan method placed_packages() dan outside_packages() untuk memisahkan paket yang dimuat dan tidak, total_value() dan total_weight() untuk menghitung agregat, value() sebagai alias total_value(), is_feasible() yang memanggil check_feasible() secara internal, serta copy() yang menghasilkan deep copy agar mutasi pada state tetangga tidak mengubah state asli. Serialisasi ke JSON dilakukan melalui to_dict() dan deserialisasi melalui from_dict().

```
class State:
    def __init__(self, truck: Truck, packages: List[Package]) -> None:
        self.truck = truck
        self.packages: List[Package] = list(packages)
        self.placements: Dict[str, Optional[Position]] = {
            pkg.id: (None if pkg.position is None else tuple(pkg.position))
            for pkg in self.packages
        }
        for pkg in self.packages:
            pkg.position = self.placements[pkg.id]

    def set_position(self, pkg_id: str, position: Optional[Position]) ->
None:
        self.placements[pkg_id] = position
        self.package_by_id(pkg_id).position = self.placements[pkg_id]
```

Kelas Result (result.py) berfungsi sebagai wadah penampung seluruh data output eksperimen: nama algoritma, varian, parameter, seed, initial state, final state, history nilai objektif per iterasi, durasi, jumlah iterasi, dan metrik-metrik tambahan yang spesifik per algoritma. File ini juga menyediakan kelas pembantu Stopwatch sebagai context manager untuk mengukur durasi eksekusi.

```
@dataclass
class Result:
    algorithm: str
    variant: Optional[str] = None
    params: Dict[str, Any] = field(default_factory=dict)
    seed: Optional[int] = None
```

```
    initial_state: Optional[State] = None
    final_state: Optional[State] = None
    initial_objective: float = 0.0
    final_objective: float = 0.0
    history: List[float] = field(default_factory=list)
    duration: float = 0.0
    n_iterations: int = 0
    metrics: Dict[str, Any] = field(default_factory=dict)
```

2.1.2. Constraint dan Fungsi Objektif (src/constraint dan src/objective)

Modul constraints (constraints.py) mengimplementasikan seluruh aturan validitas yang harus dipenuhi oleh sebuah state. Helper _dims_xyz() mengambil dimensi efektif paket sesuai orientasi, dan _xyz_position() mengambil posisi paket dengan assertion. Fungsi in_bounds() memastikan paket berada sepenuhnya dalam dimensi truk. Fungsi boxes_overlap() mendeteksi tumpang tindih 3D dengan mengecek interval overlap pada ketiga sumbu secara simultan melalui helper _rect_overlap(). Fungsi _footprints_overlap() melakukan pengecekan overlap hanya pada bidang horizontal (sumbu x dan y), digunakan sebagai basis deteksi dukungan vertikal.

```
def boxes_overlap(a: Package, b: Package) -> bool:
    if a.position is None or b.position is None:
        return False
    ax, ay, az = a.position
    bx, by, bz = b.position
    aw, al, ah = _dims_xyz(a)
    bw, bl, bh = _dims_xyz(b)
    return (
        _rect_overlap(ax, ax + aw, bx, bx + bw)
        and _rect_overlap(ay, ay + al, by, by + bl)
        and _rect_overlap(az, az + ah, bz, bz + bh)
    )
```

Fungsi has_support() memeriksa apakah paket ditopang oleh lantai (z = 0) atau oleh paket lain yang berada tepat di bawahnya, menggunakan supporting_packages() yang mencocokkan ketinggian atas paket penopang dengan ketinggian bawah paket yang diperiksa serta footprint overlap secara horizontal. Fungsi supported_packages() melakukan hal sebaliknya, yaitu menemukan paket-paket yang berada tepat di atas suatu paket. Fungsi fragile_supports() memanfaatkan supported_packages() untuk mendeteksi jika ada paket yang diletakkan di atas paket fragile. Fungsi capacity_ok() memvalidasi total berat muatan terhadap kapasitas truk. Fungsi overlap_pairs() mengembalikan seluruh pasangan paket yang saling tumpang tindih.

```
def has_support(pkg: Package, others: List[Package], truck) -> bool:
    if pkg.position is None:
        return True
    _, _, zi = pkg.position
    if zi == 0:
        return True
    return len(supporting_packages(pkg, others)) > 0
```

Seluruh constraint diintegrasikan dalam fungsi check_feasible() yang mengembalikan tuple (bool, List[str]) berisi status feasible beserta daftar kode pelanggaran spesifik. Terdapat lima kategori pelanggaran: C1_bounds (paket keluar batas truk), C5_overlap (tumpang tindih antar paket), C2_floating (paket melayang tanpa penopang), C3_fragile (ada paket di atas paket fragile), dan C4_capacity (berat melebihi kapasitas).

```
def check_feasible(state: State) -> Tuple[bool, List[str]]:
    violations: List[str] = []
    truck = state.truck
    inside = state.placed_packages()
    for pkg in inside:
        if not in_bounds(pkg, truck):
            violations.append(f"C1_bounds:{pkg.id}")
    for a_id, b_id in overlap_pairs(state):
        violations.append(f"C5_overlap:{a_id}|{b_id}")
    for pkg in inside:
        if not has_support(pkg, inside, truck):
            violations.append(f"C2_floating:{pkg.id}")
    for pkg in inside:
        if pkg.is_fragile:
            for upper in fragile_supports(pkg, inside):
                violations.append(f"C3_fragile:{pkg.id}>{upper.id}")
    if not capacity_ok(state):
        violations.append("C4_capacity")
    return (len(violations) == 0), violations
```

Fungsi objektif (objective.py) mengembalikan total value jika state feasible, atau penalti negatif sebesar jumlah pelanggaran jika tidak, sehingga memastikan algoritma terdorong menuju ruang solusi yang valid. Tersedia juga fungsi feasible_value() yang mengembalikan total value tanpa pengecekan feasibility, digunakan untuk keperluan pengujian.

```
def objective(state: State) -> float:
    feasible, violations = check_feasible(state)
    return state.total_value() if feasible else -float(len(violations))
```

2.1.3. Operator Pergerakan dan Inisialisasi (src/moves dan src/init)

Modul moves (moves.py) menyediakan tiga operator pergerakan: relocate() memindahkan satu paket ke posisi baru atau None (mengeluarkan dari truk); swap() menukar posisi dua paket; rotate_90() merotasi satu paket 90° terhadap salah satu sumbu (x, y, atau z).

```
def relocate(state: State, pkg_id: str, position: Optional[Position]) ->
State:
    new = state.copy()
    new.set_position(pkg_id, position)
    return new

def swap(state: State, id_a: str, id_b: str) -> State:
    new = state.copy()
    pos_a, pos_b = new.placements[id_a], new.placements[id_b]
    new.set_position(id_a, pos_b)
    new.set_position(id_b, pos_a)
    return new

def rotate_90(state: State, pkg_id: str, axis: str) -> State:
    new = state.copy()
    pkg = new.package_by_id(pkg_id)
    pkg.rotate(axis)
    return new
```

Posisi kandidat dihasilkan oleh candidate_positions() yang men-generate posisi di lantai truk (z=0) dan di atas paket-paket yang sudah ada (posisi stacking). Posisi sudut truk juga ditambahkan secara eksplisit sebagai heuristik. Hasilnya diacak dan dibatasi sebanyak parameter n.

Fungsi neighbor_states() menggabungkan ketiga operator secara acak untuk membangkitkan sejumlah state tetangga. Pada setiap percobaan, salah satu tipe move dipilih secara random. Untuk move relocate, terdapat probabilitas 0.3 untuk mengeluarkan paket dari truk (set posisi ke None), dan 0.7 untuk memindahkan ke posisi kandidat baru. Duplikat dicegah melalui himpunan seen_positions, dan secara default hanya state feasible yang diloloskan (filter_feasible=True). Jumlah percobaan dibatasi hingga n_neighbors * 10 untuk menghindari loop tak terbatas.

```
def neighbor_states(state, rng, n_neighbors=60, grid_step=1,
                    use_swap=True, use_relocate=True, use_rotate=True,
                    filter_feasible=True) -> List[State]:
    ...
    while len(neighbors) < n_neighbors and attempts < max_attempts:
        move_type = rng.choice(move_pool)
        ...
        if filter_feasible and not cand.is_feasible():
```

```
            continue
        neighbors.append(cand)
    return neighbors
```

Inisialisasi initial state (init.py) dilakukan secara random oleh random_initial_state(). Fungsi ini pertama-tama membuat salinan seluruh paket dan mereset posisi ke None, kemudian mengacak urutan paket. Untuk setiap paket, dicoba beberapa orientasi random (orientasi asli ditambah 2 orientasi hasil rotasi acak), lalu menempatkan paket ke posisi kandidat sambil memvalidasi feasibility. Paket yang tidak berhasil ditempatkan dibiarkan di luar truk (None).

```
def random_initial_state(truck, packages, rng, grid_step=1,
n_candidates=12) -> State:
    state = State(truck, packages)
    state.packages = [p.copy() for p in packages]
    state.placements = {p.id: None for p in state.packages}
    for p in state.packages:
        p.position = None
    order = list(state.packages)
    rng.shuffle(order)
    for pkg in order:
        orientations = [dict(pkg.orient)]
        for _ in range(2):
            axis = rng.choice(("x", "y", "z"))
            trial = pkg.copy()
            trial.rotate(axis)
            orientations = orientations + [dict(trial.orient)]
        placed = False
        for orient in orientations:
            if placed:
                break
            pkg.set_orientation(orient)
            positions = candidate_positions(state, pkg.id, rng,
n=n_candidates)
            rng.shuffle(positions)
            for pos in positions:
                state.set_position(pkg.id, pos)
                feasible, _ = check_feasible(state)
                if feasible:
                    placed = True; break
                state.set_position(pkg.id, None)
    return state
```

2.1.4. Hill Climbing (src/algortima/hill_climbing)

Algoritma Hill Climbing (hill_climbing.py) diimplementasikan dengan empat varian melalui dataclass HCParams. Parameter utama mencakup max_iterations (200), n_neighbors (60 tetangga per iterasi), max_sideways (30 langkah lateral), sideways_prob (0.1 untuk stochastic), max_restart (5 kali restart), dan iterations_per_restart (100 iterasi per segmen restart).

```
VARIANTS: Tuple[str, ...] = ("steepest", "sideways", "restart",
"stochastic")

@dataclass
class HCParams:
    variant: str = "steepest"
    max_iterations: int = 200
    n_neighbors: int = 60
    grid_step: int = 1
    max_sideways: int = 30
    sideways_prob: float = 0.1
    max_restart: int = 5
    iterations_per_restart: int = 100
    no_improvement_limit: Optional[int] = None
```

Setiap varian diimplementasikan sebagai fungsi terpisah, bukan satu fungsi bersama:

Steepest Ascent (_steepest_ascent): Pada setiap iterasi, seluruh tetangga dievaluasi dan tetangga terbaik dipilih. Hanya menerima langkah jika nilainya lebih besar dari current. Jika tidak ada perbaikan, pencarian langsung berhenti di local optimum (break). Fungsi ini mengembalikan tuple berisi state, best objective, history, jumlah iterasi, 0 (tidak ada sideways), dan jumlah improving moves.

```
# Steepest Ascent
best_neighbor_obj, best_neighbor = max(scored, key=lambda t: t[0])
if best_neighbor_obj > current_obj:
    current, current_obj = best_neighbor.copy(), best_neighbor_obj
    best_obj = current_obj
    improving_count += 1
else:
    break  # local optimum
```

Sideways Move (_sideways_move): Memperluas Steepest Ascent dengan mengizinkan perpindahan ke tetangga yang bernilai sama (plateau) hingga max_sideways kali (default 30). Pencarian berhenti ketika: (a) kuota sideways habis, (b) tidak ada langkah yang diterima, atau (c) no_improvement_limit tercapai.

```
# Sideways Move
if best_neighbor_obj > current_obj:
    current, current_obj = best_neighbor.copy(), best_neighbor_obj
    improving_count += 1
elif best_neighbor_obj == current_obj and sideways_used <
params.max_sideways:
    current, current_obj = best_neighbor.copy(), best_neighbor_obj
    sideways_used += 1
```

Stochastic (_stochastic): Memilih secara acak di antara tetangga-tetangga yang lebih baik. Jika tidak ada tetangga yang lebih baik, dengan probabilitas sideways_prob (0.1) dipilih secara acak satu tetangga dengan nilai yang sama (sideways move). Pencarian berhenti jika tidak ada langkah yang diterima sama sekali.

```
# Stochastic
improving = [(o, nb) for o, nb in scored if o > current_obj]
if improving:
    next_obj, next_state = rng.choice(improving)
    current, current_obj = next_state.copy(), next_obj
elif rng.random() < params.sideways_prob:
    equal = [(o, nb) for o, nb in scored if o == current_obj]
    if equal:
        next_obj, next_state = rng.choice(equal)
        current, current_obj = next_state.copy(), next_obj
```

Random Restart (_random_restart): Menjalankan _sideways_move() berulang (hingga max_restart + 1 kali, yaitu 6 segmen total). Setiap segmen menggunakan iterations_per_restart (100) sebagai batas iterasi. Initial state baru dibangkitkan melalui _random_restart_state() yang memanggil random_initial_state(). Solusi terbaik global dilacak sepanjang seluruh segmen.

```
for r in range(params.max_restart + 1):
    if r == 0:
        start_state = initial_state.copy()
    else:
        start_state = _random_restart_state(initial_state, rng)
        n_restarts += 1
    seg_params = replace(params,
max_iterations=params.iterations_per_restart)
    seg_state, seg_obj, seg_hist, seg_iters, seg_side, seg_imp =
_sideways_move(
        start_state, seg_params, rng
    )
    if seg_obj > best_obj:
        best_obj, best_state = seg_obj, seg_state
```

Setiap eksperimen Hill Climbing mencatat metrik: variant, sideways_moves, restarts, iterations_per_restart (distribusi iterasi per segmen), dan improving_moves.

2.1.5. Simulated Annealing (src/algoritma/simulated_annealing)

SA (simulated_annealing.py) diimplementasikan dengan parameter temperatur awal 100.0, temperatur minimum 0.01, cooling rate 0.995 (jadwal eksponensial), dan maksimum 1000 iterasi. Pada setiap iterasi, satu tetangga di-generate (n_neighbors=1), dan keputusan penerimaan didasarkan pada tiga kategori: (a) jika

delta_e > 0 (tetangga lebih baik), langsung diterima; (b) jika delta_e == 0 (nilai sama), langsung diterima dan dicatat sebagai accepted_equal; (c) jika delta_e < 0 (tetangga lebih buruk), diterima dengan probabilitas e^(ΔE/T). Jika tidak ada tetangga feasible yang ditemukan dalam satu iterasi, terdapat mekanisme retry hingga max_no_neighbor_retries (100 kali).

```
@dataclass
class SAParams:
    max_iterations: int = 1000
    n_neighbors: int = 1
    grid_step: int = 1
    temp_initial: float = 100.0
    temp_min: float = 0.01
    cooling_rate: float = 0.995
    schedule: str = "exponential"
    max_no_neighbor_retries: int = 100
```

```
delta_e = next_obj - current_obj
exp_val = math.exp(delta_e / T)

if delta_e > 0:
    current = next_state.copy()
    metrics["improving_moves"] += 1
elif delta_e == 0:
    current = next_state.copy()
    metrics["accepted_equal"] += 1
else:
    if rng.random() < exp_val:
        current = next_state.copy()
        metrics["accepted_worse"] += 1
    else:
        metrics["rejected"] += 1

# Penurunan temperatur
if params.schedule == "exponential":
    T *= params.cooling_rate
```

Metrik yang dicatat mencakup accepted_equal, accepted_worse, rejected, improving_moves, stuck_count (iterasi tanpa kemajuan), no_neighbor_steps, final_temp, prob_history (seluruh nilai e^(ΔE/T) per iterasi), dan current_history (nilai objektif state saat ini per iterasi, berbeda dari history yang melacak best).

2.1.6. Genetic Algorithm (src/algoritma/genetic_algorithm)

GA (genetic_algorithm.py) diimplementasikan dengan populasi awal yang terdiri dari initial state asli ditambah pop_size - 1 individu random (via random_initial_state()). Setiap generasi menerapkan:

- Elitisme (2 individu terbaik langsung masuk generasi berikutnya)
- Seleksi (roulette wheel dengan shift untuk menangani nilai negatif)
- Crossover (satu titik potong; gen setelah titik potong diambil dari parent B dengan validasi feasibility per paket; rollback jika tidak feasible)
- Mutasi (probabilitas 0.1; menggunakan neighbor_states() dengan n_neighbors=15 untuk menghasilkan varian feasible)

```
@dataclass
class GAParams:
    pop_size: int = 20
    max_generations: int = 100
    mutation_rate: float = 0.1
    n_neighbors: int = 15
    grid_step: int = 1
    elitism: int = 2
    no_improvement_limit: Optional[int] = None
```

```
# Crossover satu titik potong
cut = rng.randint(1, len(all_ids) - 1)
child = parent_a.copy()
for pkg_id in all_ids[cut:]:
    child_pkg.set_orientation(dict(pkg_b.orient))
    child.set_position(pkg_id, parent_b.placements.get(pkg_id))
    feasible, _ = check_feasible(child)
    if not feasible:  # rollback jika tidak feasible
        child_pkg.set_orientation(orig_orient)
        child.set_position(pkg_id, orig_pos)
```

GA juga melacak avg_history yang mencatat rata-rata nilai objektif populasi per generasi, selain history yang mencatat best per generasi. Metrik tambahan meliputi crossovers, mutations, dan improving_generations.

2.1.7. Sistem Runner dan Konfigurasi (src/main dan src/config)

Modul main.py mengorkestrasikan seluruh eksperimen secara otomatis. Test case ini selanjutnya akan dibangkitkan oleh generate_test_case() menggunakan random.seed(RANDOM_SEED) untuk memastikan determinisme: truk berdimensi 14 × 24 × 12 dengan kapasitas berat 450, dan 40 paket acak. Setiap paket memiliki dimensi 2–6 per sisi, nilai 10–100, berat 5–50, dan probabilitas fragile 20%.

Parameter eksperimen didefinisikan di config.py:

```
RANDOM_SEED = 317009
NUM_RUNS_STANDARD = 3

GA_CONST_POPULATION = 20
```

```
GA_ITERATION_VARS = [50, 100, 150]
GA_CONST_ITERATIONS = 100
GA_POPULATION_VARS = [10, 20, 30]
```

Setiap algoritma HC (4 varian) dan SA dijalankan NUM_RUNS_STANDARD = 3 kali (total 15 run). GA memiliki dua skenario variasi: (a) populasi konstan 20 dengan iterasi [50, 100, 150] masing-masing 3 kali (9 run), dan (b) iterasi konstan 100 dengan populasi [10, 20, 30] masing-masing 3 kali (9 run). Total: 33 eksperimen. Seluruh hasil disimpan sebagai file JSON ke folder output/. Modul helpers.py menyediakan utilitas _now() (wrapper time.perf_counter() untuk pengukuran durasi presisi tinggi) dan asdict_safe() untuk konversi parameter ke dictionary secara aman.

2.2. Hasil Eksperimen

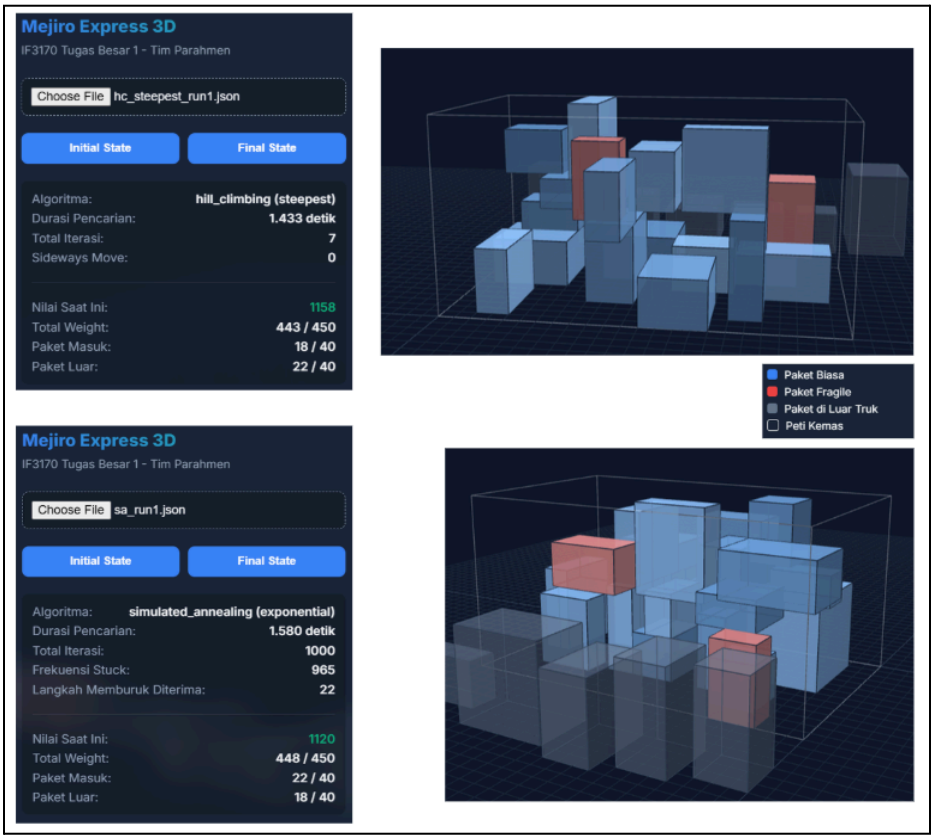
Eksperimen dijalankan dengan test case berupa truk berdimensi {w:14, l:24, h:12} dengan kapasitas berat 450, dan 40 paket acak (seed tetap = 317009). Paket memiliki dimensi 2–6, nilai 10–100, berat 5–50, dan probabilitas *fragile* sebesar 20%. Initial state yang di-generate secara random oleh random_initial_state() memiliki nilai objektif awal sebesar 849,0, dengan hanya sebagian paket berhasil ditempatkan di dalam truk. Karena batas kapasitas berat bersifat ketat (*binding*), total berat 40 paket jauh melebihi kapasitas truk sebesar 450, sistem harus menyeleksi subset paket secara selektif. Kondisi ini memberikan ruang perbaikan yang signifikan bagi algoritma *local search* untuk menemukan kombinasi paket yang lebih optimal. Total terdapat 33 eksperimen yang dijalankan.

2.2.1. Sistem Runner dan Konfigurasi (src/main dan src/config)

Tabel 2.1. Detail Hasil Eksperimen Algoritma Hill Climbing dan Simulated Annealing.

Algoritma	Run	Obj. Awal	Obj. Akhir	Iterasi	Durasi (detik)	Improving Moves	Sideways Moves
HC Steepest Ascent	1	849	1158	7	1,43	6	0
HC Steepest Ascent	2	849	1224	8	1,70	7	0
HC Steepest Ascent	3	849	1211	10	2,15	9	0

HC Sideways Move	1	849	1233	41	9,16	11	30
HC Sideways Move	2	849	1208	40	8,97	10	30
HC Sideways Move	3	849	1243	42	9,43	12	30
HC Random Restart	1	849	1248	245	54,40	—	—
HC Random Restart	2	849	1247	238	54,60	—	—
HC Random Restart	3	849	1235	246	57,00	—	—
HC Stochastic	1	849	1136	11	2,40	—	—
HC Stochastic	2	849	1189	14	2,97	—	—
HC Stochastic	3	849	1122	12	2,60	—	—
Simulated Annealing	1	849	1120	1000	1,58	35	—
Simulated Annealing	2	849	1143	1000	1,25	22	—
Simulated Annealing	3	849	1119	1000	1,36	28	—



Gambar 2.1. "Salah satu contoh kasus visualisasi 3D konfigurasi final state hasil eksperimen Hill Climbing Steepest Ascent And Simulated Annealing Run 1.

2.2.2. Hasil Eksperimen Genetic Algorithm

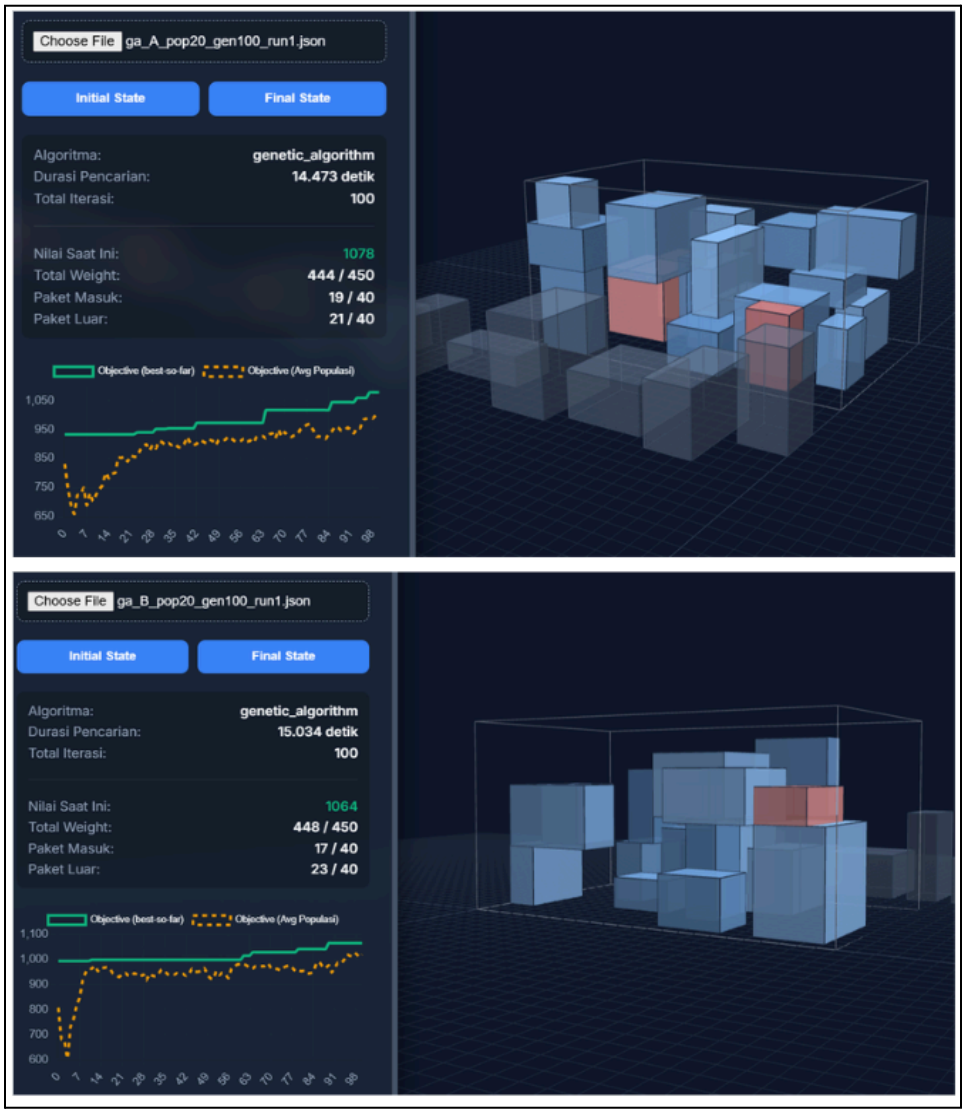
Tabel 2.2. *Hasil Eksperimen Skenario A Genetic Algorithm (Populasi konstan* (20), variasi iterasi*)*

Iterasi	Run	Obj. Awal	Obj. Akhir	Durasi (detik)
50	1	849	1077	8,11
50	2	849	1114	8,58
50	3	849	1092	8,78
100	1	849	1078	14,47
100	2	849	1195	17,27
100	3	849	1112	15,77
150	1	849	1222	22,54

150	2	849	1206	21,91
150	3	849	1119	22,65

Tabel 2.3. *Hasil Eksperimen Skenario B Genetic Algorithm (Iterasi konstan* (100), variasi populasi*)*

Populasi	Run	Obj. Awal	Obj. Akhir	Durasi (detik)
10	1	849	990	7,66
10	2	849	1146	8,08
10	3	849	1080	7,74
20	1	849	1064	15,03
20	2	849	1146	12,37
20	3	849	1094	11,60
30	1	849	1165	21,01
30	2	849	1160	22,43
30	3	849	1077	20,64



Gambar 2.2. Contoh kasus plot nilai e^(ΔE/T) (kasus A dan kasus B) terhadap iterasi pada Genetic Algorithm Run 1.

2.3. Analisa Hasil Eksperimen

2.3.1. Ringkasan Performa per Algoritma

Tabel 2.4. *Hasil Eksperimen Seluruh Bagian dengan algoritma masing-masing*

Algoritma	Rata-rata Obj. Akhir	Rentang	Rata-rata Durasi (Detik)
HC Random Restart	1243,3	1235 - 1248	55,33

HC Sideways Move	1228,0	1208 - 1243	9,19
HC Steepest Ascent	1197,7	1158 - 1224	1,76
HC Stochastic	1149,0	1122 - 1189	2,66
Simulated Annealing	1127,3	1119 - 1143	1,40
Genetic Algorithm (semua)	1118,7	1119 - 1143	14,81

Seluruh 33 eksperimen menghasilkan solusi akhir yang valid (feasible) dalam rentang nilai 990 hingga 1248, menunjukkan peningkatan signifikan dari initial state 849.

2.3.2. Analisis Kualitas Solusi

Hill Climbing Random Restart mencatat performa tertinggi dan paling stabil dengan rata-rata 1243,3 (rentang sempit 1235–1248, standar deviasi ≈ 7,2). Keberhasilan varian ini disebabkan oleh kemampuan eksplorasi multi-titik: menjalankan 6 segmen pencarian independen dengan initial state berbeda, lalu memilih solusi terbaik. Setiap segmen menggunakan mekanisme sideways move internal, sehingga juga mampu menavigasi plateau.

Hill Climbing Sideways Move berada di posisi kedua dengan rata-rata 1228,0 (rentang 1208–1243). Dibandingkan Steepest Ascent, penambahan kemampuan sideways move (masing-masing run menggunakan tepat 30 sideways moves) memungkinkan algoritma menavigasi plateau dan menemukan “celah" menuju puncak yang lebih tinggi. Jumlah iterasi berkisar 40–42, jauh lebih banyak dibanding Steepest Ascent (7–10), menunjukkan bahwa sideways moves secara efektif memperpanjang pencarian.

Hill Climbing Steepest Ascent meraih rata-rata 1197,7 (rentang 1158–1224). Algoritma ini berhenti sangat cepat (7–10 iterasi) begitu mencapai local optimum pertama di mana tidak ada tetangga feasible yang bernilai lebih tinggi. Meskipun demikian, hasilnya tetap kompetitif karena setiap iterasi secara greedy memilih tetangga terbaik dari 60 kandidat.

Hill Climbing Stochastic memperoleh rata-rata 1149,0 (rentang 1122–1189). Varian ini menunjukkan variabilitas yang lebih tinggi dibanding Steepest Ascent, karena elemen keacakan dalam pemilihan tetangga. Ironisnya, rata-ratanya lebih

rendah, keacakan pemilihan tidak selalu menguntungkan pada landscape ini, karena memilih tetangga yang lebih baik secara random (bukan terbaik) menghasilkan lintasan yang kurang optimal.

Simulated Annealing menghasilkan rata-rata 1127,3 (rentang 1119–1143). Meskipun SA mengeksekusi penuh 1000 iterasi, performanya berada di bawah HC. Hal ini disebabkan oleh beberapa faktor:

- SA hanya mengevaluasi 1 tetangga per iterasi, berbeda dengan HC yang mengevaluasi 60.
- Penurunan temperatur eksponensial (0.995) menyebabkan suhu akhir mencapai 0,665, masih cukup tinggi untuk sesekali menerima langkah buruk di akhir pencarian, tetapi tidak cukup rendah untuk sepenuhnya konvergen.
- Metrik menunjukkan stuck_count yang sangat tinggi (965 - 978 dari 1000 iterasi), artinya SA sangat sering menemui kondisi di mana tetangga tunggalnya tidak lebih baik dari state saat ini.

Genetic Algorithm mencatat rata-rata 1118,7 dengan variabilitas terbesar (rentang 990 - 1222). Fluktuasi ini menunjukkan bahwa GA sangat sensitif terhadap keacakan: crossover sering menghasilkan individu infeasible (yang di-rollback), sementara mutasi terbatas pada 15 tetangga. Meskipun demikian, GA mampu mencapai 1222 pada skenario terbaik (150 generasi), mendekati performa HC Sideways.

2.3.3. Analisis Durasi dan Tradeoff Kualitas Waktu

Tabel 2.5. *Durasi dan Tradeoff kualitas waktu per algoritma*

Algoritma	Rata-rata Durasi (Detik)	Rata-rata Obj.	Efisiensi
Simulated Annealing	1,40	1127,3	805,2
HC Steepest Ascent	1,76	1197,7	680,5
HC Stochastic	2,66	1149,0	432,0
HC Sideways Move	9,19	1228,0	133,6
GA (semua rata-rata)	14,81	1118,7	75,5

HC Random Restart	55,33	1243,3	22,5

- HC Steepest Ascent menawarkan efisiensi hasil per satuan waktu terbaik secara praktis, meraih nilai rata-rata 1197,7 hanya dalam 1,76 detik, peningkatan 348,7 poin dari initial state 849 dalam waktu sangat singkat.
- Simulated Annealing mencatat waktu eksekusi tercepat (1,40 detik), namun kualitas solusinya lebih rendah. SA memproses 1000 iterasi sangat cepat karena hanya mengevaluasi 1 tetangga per iterasi, tetapi cenderung terjebak pada plateau lokal karena kurangnya eksplorasi per langkah.
- HC Sideways Move membutuhkan ~9,19 detik untuk peningkatan kualitas signifikan (+30 poin di atas Steepest, rata-rata), menjadikannya pilihan yang baik jika durasi beberapa detik dapat ditoleransi.
- HC Random Restart memerlukan waktu terlama (~55 detik), tetapi terbukti paling efektif dalam mengeksplorasi ruang pencarian dan menemukan solusi berkualitas tertinggi.
- Genetic Algorithm memerlukan rata-rata 14,81 detik dengan kualitas rata-rata paling rendah, menunjukkan overhead yang tinggi dari operasi crossover, mutasi, dan evaluasi populasi.

2.3.4. Analisis Probabilitas pada Simulated Annealing

Data metrik Simulated Annealing menunjukan pola yang konsisten di ketiga run:

Tabel 2.5. *Detail Probabilitas algoritma Simulated Annealing*

Metrik	Run 1	Run 2	Run 3
Improving Moves	35	22	28
Accepted Equal (ΔE = 0)	844	866	851
Accepted Worse (ΔE < 0)	22	19	18
Rejected	99	93	103
Stuck Count	965	978	972
No-Neighbor Steps	11	5	3
Final Temperature	0,665	0,665	0,665

Mayoritas iterasi (~85%) menghasilkan tetangga dengan nilai yang sama (accepted_equal), menunjukkan bahwa SA lebih sering "berjalan di tempat" pada plateau daripada melakukan eksplorasi aktif. Data prob_history pada Run 1 menunjukkan bahwa:

- Sebagian besar nilai e^(ΔE/T) adalah 1,0, yaitu ketika tetangga memiliki nilai yang sama dengan state saat ini (ΔE = 0).
- Nilai di bawah 1,0 (rentang ~0,002–0,85) muncul ketika tetangga lebih buruk, semakin rendah temperatur, semakin kecil probabilitas penerimaan.
- Nilai di atas 1,0 (hingga ~34,9) muncul ketika ΔE > 0, mengindikasikan tetangga yang lebih baik. Nilai-nilai besar ini muncul di iterasi akhir karena pembaginya (T) sudah kecil.

Seluruh run mencapai suhu akhir yang sama (0,665) karena cooling rate dan jumlah iterasi identik. Suhu ini belum turun di bawah temp_min (0,01), sehingga SA menyelesaikan penuh 1000 iterasi tanpa early stopping akibat suhu.

2.3.5. Analisis Pengaruh Parameter pada Genetic Algorithm

Pengaruh jumlah generasi (Skenario A, populasi=20):

- 50 generasi: rata-rata 1094,3, durasi rata-rata 8,49 detik
- 100 generasi: rata-rata 1128,3, durasi rata-rata 15,84 detik
- 150 generasi: rata-rata 1182,3, durasi rata-rata 22,37 detik

Peningkatan jumlah generasi meningkatkan kualitas solusi secara konsisten (+88 poin dari 50 ke 150 generasi), tetapi durasi meningkat hampir linear (~0,15 detik per generasi tambahan). Nilai tertinggi (1222) tercapai pada 150 generasi.

Pengaruh ukuran populasi (Skenario B, generasi=100):

- Populasi 10: rata-rata 1072,0, durasi rata-rata 7,83 detik
- Populasi 20: rata-rata 1101,3, durasi rata-rata 13,00 detik
- Populasi 30: rata-rata 1134,0, durasi rata-rata 21,36 detik

Penambahan ukuran populasi juga meningkatkan kualitas, meskipun peningkatannya lebih moderat (+62 poin dari populasi 10 ke 30). Durasi meningkat hampir linear terhadap ukuran populasi. Variabilitas yang tinggi (rentang per konfigurasi bisa mencapai ~100 poin) menunjukkan bahwa GA memerlukan ukuran populasi dan

jumlah generasi yang jauh lebih besar untuk mencapai konvergensi stabil pada permasalahan ini.

2.3.6. Analisis Pengaruh Parameter pada Genetic Algorithm

Perbandingan jumlah iterasi yang dijalankan per algoritma menunjukkan perbedaan signifikan:

- HC Steepest Ascent: 7–10 iterasi, konvergen sangat cepat ke local optimum terdekat.
- HC Sideways Move: 40–42 iterasi, kemampuan sideways memperpanjang pencarian secara signifikan (improving moves 10–12, ditambah 30 sideways moves).
- HC Stochastic: 11–14 iterasi, sedikit lebih lama dari steepest, tetapi keacakan tidak memberikan keuntungan konsisten.
- HC Random Restart: 238–246 iterasi total (tersebar di 6 segmen), eksplorasi paling luas.
- SA: 1000 iterasi penuh, tetapi hanya ~2–3,5% iterasi menghasilkan improvement (improving moves 22–35), sisanya adalah langkah sideways atau stagnasi.
- GA: 50–150 generasi sesuai konfigurasi, dengan setiap generasi melibatkan evaluasi seluruh populasi.

3. Kesimpulan dan Saran

3.1. Kesimpulan

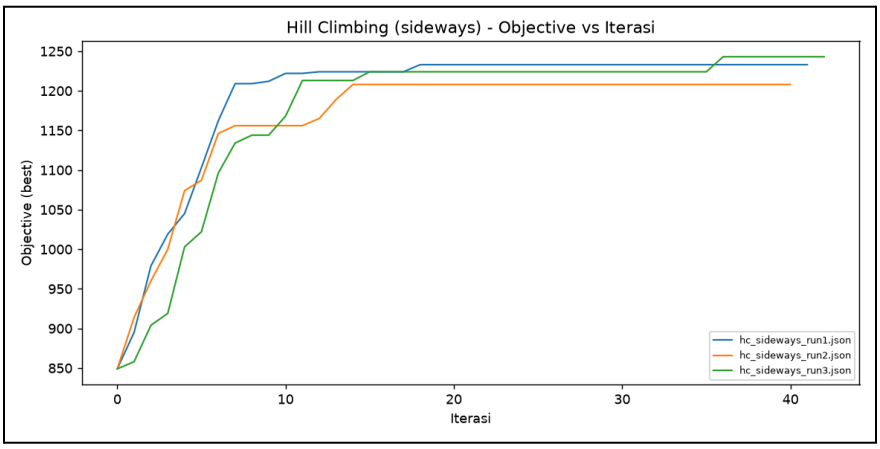
Tugas ini menerapkan pendekatan local search untuk menyelesaikan permasalahan 3D Truck Loading Optimization dengan memaksimalkan total nilai paket di dalam kontainer tanpa melanggar kendala fisik. Kendala tersebut mencakup batas dimensi ruang, larangan tumpang tindih (*overlapping*), dukungan alas (*bottom support*), aturan fragile (paket rentan tidak boleh ditindih), serta batas kapasitas berat. Model mengintegrasikan representasi fisik truk dan paket, *feasibility check*, fungsi objektif, operator tetangga (swap, shift, dan rotasi 90°), dan inisialisasi acak. Tiga algoritma yang diuji yaitu Hill Climbing (Steepest Ascent, Sideways Move, Stochastic, dan Random Restart), Simulated Annealing (SA), dan Genetic Algorithm (GA). Pengujian dilakukan secara deterministik pada satu instans menggunakan seed 317009 (dimensi truk 14 × 24 × 12, batas berat 450, dan 40 kandidat paket). Karena batas kapasitas berat bersifat ketat (binding), sistem menyeleksi subset paket secara selektif. Dari nilai awal sebesar 849, seluruh algoritma menghasilkan solusi akhir yang valid dalam rentang 990 hingga 1248 di seluruh 33 file output.

Hasil Pengujian

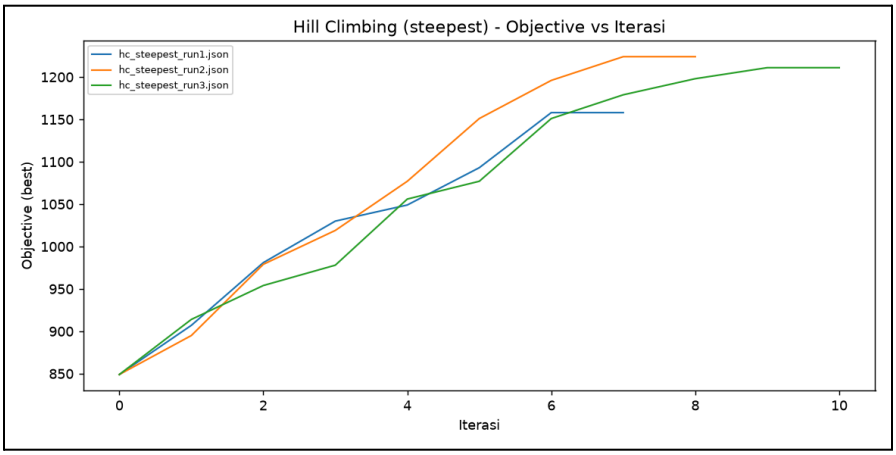
- Hill Climbing Random Restart mencatatkan performa tertinggi dan paling stabil dengan rata-rata nilai 1243,3 (rentang 1235 - 1248).



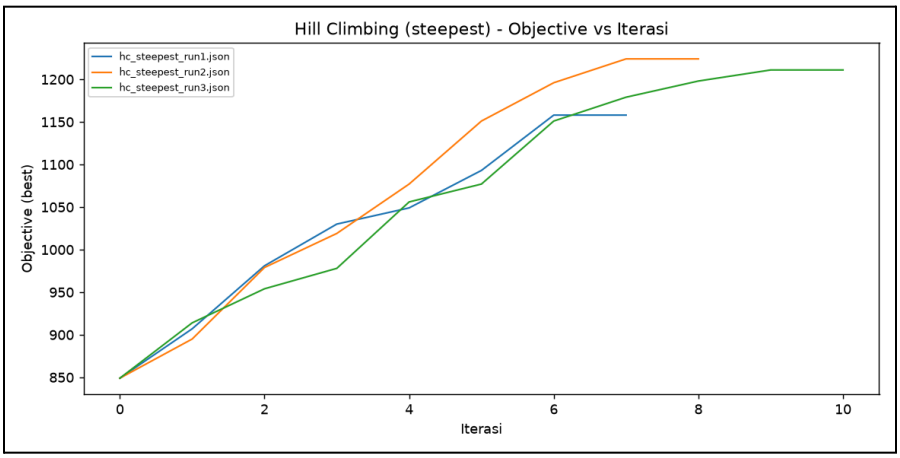
- Hill Climbing Sideways Move berada di posisi kedua dengan rata-rata 1228,0 (rentang 1208 - 1243).



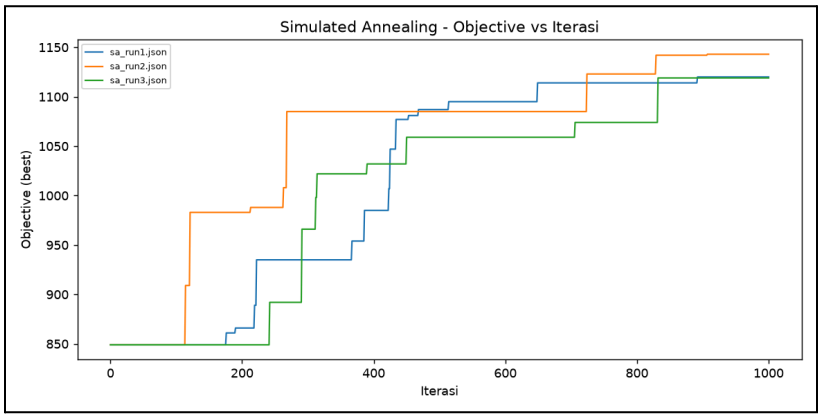
- Hill Climbing Steepest Ascent meraih rata-rata 1197,7 (rentang 1158 - 1224).



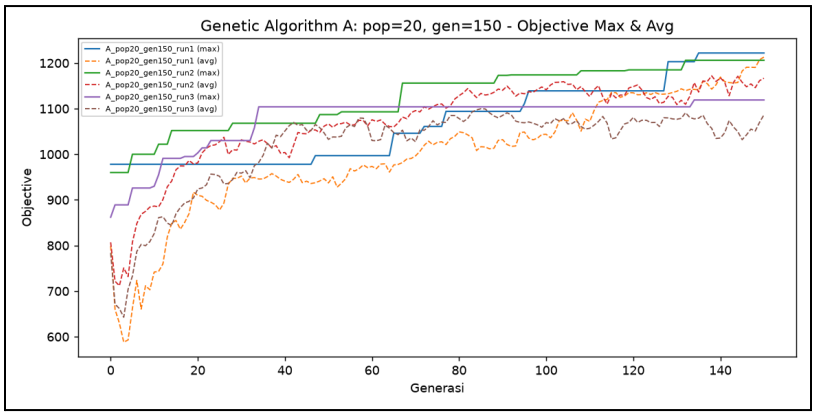
- Hill Climbing Stochastic memperoleh rata-rata 1149,0 (rentang 1122 - 1189).



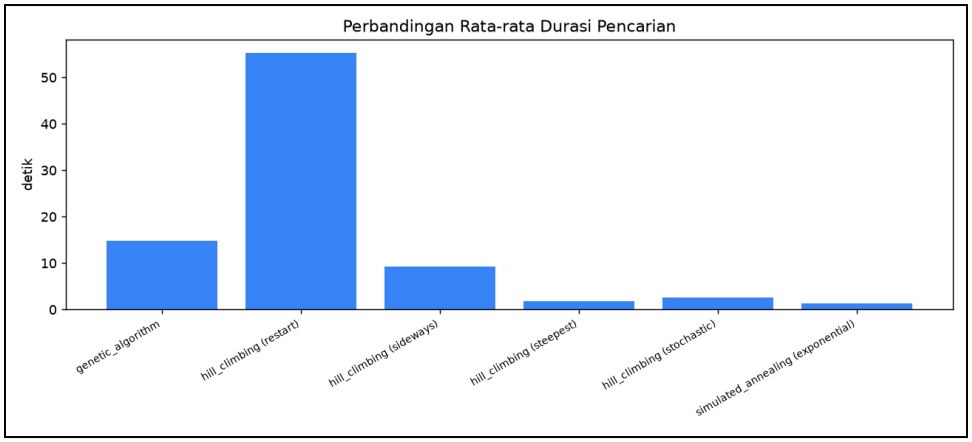
- Simulated Annealing menghasilkan rata-rata 1127,3 (rentang 1119 - 1143).



- Genetic Algorithm mencatat rata-rata 1118,7 dengan variabilitas sebaran terbesar (rentang 990 - 1222).



Tradeoff Kualitas dan Efisiensi Waktu



- Hill Climbing Steepest Ascent menawarkan efisiensi hasil per satuan waktu terbaik, meraih nilai 1197,7 hanya dalam 1,76 detik.
- Simulated Annealing mencatatkan waktu eksekusi tercepat (1,40 detik), namun cenderung terjebak pada optimum lokal karena penurunan suhu belum optimal (suhu akhir 0,665 pada iterasi 1000).
- Hill Climbing Sideways Move membutuhkan 9,19 detik untuk peningkatan kualitas solusi yang lebih tinggi.
- Hill Climbing Random Restart memerlukan waktu terlama (55,33 detik), tetapi terbukti paling efektif mengeksplorasi ruang pencarian.
- Genetic Algorithm memerlukan rata-rata 14,81 detik. Penambahan ukuran populasi dan generasi (hingga 150 generasi) berhasil menaikkan kualitas solusi hingga 1222, meski belum melampaui Random Restart.

3.2. Saran

- Perluas pengujian menggunakan berbagai variasi instans dan seed yang acak
- Reset ulang jadwal pendinginan pada algoritma SA. Pertimbangkan penambahan jumlah tetangga per iterasi, modifikasi laju penurunan suhu, skema reheating atau peningkatan total iterasi.
- Tambahkan utilitas untuk membaca kerapatan susunan, stabilitas secara fisik dan pusat gravitasi

4. Lampiran

4.1. Surat Kebijakan Penggunaan AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, Kami:
Nama/NIM : Ardell Aghna Mahendra/13523151, Fazri Arrashyi Putra/13524127, Dzakwan Muhammad K. P. P. / 13524145
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3170
Judul Tugas : Tugas Besar 1 IF3170 Artifisial Intelegensi Algoritma Local Search

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	3 - AI-assisted workflow
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: Menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1 - No AI
Lainnya, Jelaskan:		

Kami Juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 17 Oktober 2026



(Ardell Aghna Mahendra)
13523151



(Fazri Arrashyi Putra)
13524127



(Dzakwan Muhammad K. P. P)
13524145

4.2. Detail Pembagian Tugas

Berikut ini merupakan pembagian tugas untuk kelompok kami:

Nama Penanggung Jawab	Detail Tugas
Ardell Aghna Mahendra (13523151)	Membuat algoritma SA, membuat algoritma GA, testing program, menambahkan visualisasi untuk algoritma SA, dan laporan
Fazri Arrashyi Putra (13524127)	Inisiasi, 4 Algoritma HC, Testing, Dokumen Laporan
Dzakwan Muhammad K. P. P. (13524145)	Rancang Web visualisasi via HTML, konstruksi main, eksekusi CLI dan ekspor output dalam format .json, inisiasi desain utama Laporan

4.3. Lampiran Lainnya

- Link canva untuk pembuatan cover dan gambar bagian 1 laporan: https://www.canva.com/design/DAHYKDngWn0/LwWgRrBhr9iASaNWuzwpmA/edit
- Link Repository: https://github.com/ArdellAghna/IF3170_G09_Parahmen

5. Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 25, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 25, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 25, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 25, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 25, 2026].

[6] Tim Asisten Laboratorium AI tahun 2023, "Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027. Available: Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027 [Accessed: Sep. 25, 2026].